

Relational Graph Transformer

(RELGT)

Báo cáo Phân tích Kỹ thuật Chuyên sâu

Đổi mới, Phương pháp, Toán học, So sánh và Đánh giá

Nguồn gốc: arXiv:2505.10960v1 [cs.LG], 16 tháng 5 năm 2025

Tác giả: Vijay Prakash Dwivedi, Sri Jaladi, Yangyi Shen,
Federico López, Charilaos I. Kanatsoulis, Rishi Puri,
Matthias Fey, Jure Leskovec

Tổ chức: Stanford University, Kumo.AI, NVIDIA

Tóm tắt

RELGT là kiến trúc Graph Transformer đầu tiên được thiết kế đặc biệt cho dữ liệu bảng quan hệ đa bảng. Mô hình vượt trội GNN cơ sở lên đến **18%** trên 21 tác vụ trong chuẩn đánh giá RelBench bằng chiến lược token hóa 5 thành phần và cơ chế chú ý cục bộ-toàn cục.

Báo cáo được tổng hợp và phân tích bởi:
Dựa trên mã nguồn `relgt-upstream` và bài báo gốc.

Ngày 23 tháng 4 năm 2026

0 Mục lục

1	Giới thiệu và Bối cảnh	3
1.1	Vấn đề đặt ra	3
1.2	Đóng góp của RELGT	3
2	Nền tảng Lý thuyết	3
2.1	Định nghĩa Relational Deep Learning	3
2.2	Đặc điểm nổi bật của REG	4
3	Phương pháp: Kiến trúc RELGT	4
3.1	Tổng quan kiến trúc	4
3.2	Giai đoạn 1: Lấy mẫu Thời gian	4
3.3	Giai đoạn 2: Token Hóa Đa Thành Phần	4
3.3.1	Biểu diễn 5-tuple	4
3.3.2	Các Bộ Mã Hóa (Encoders)	5
3.3.3	Kết hợp Token	6
3.4	Giai đoạn 3: Mạng Transformer	7
3.4.1	Mô-đun Cục bộ (Local Module)	7
3.4.2	Mô-đun Toàn cục (Global Module)	7
3.4.3	Kết hợp Cục bộ – Toàn cục	8
3.5	Đầu ra Dự đoán	8
4	Phân tích Toán học Chuyên sâu	8
4.1	Độ phức tạp Tính toán	8
4.2	Phân tích Tính biểu đạt (Expressiveness)	8
4.3	Cơ chế Phòng tránh Codebook Collapse	9
5	Các Cải tiến trong Triển khai (RelGT++)	9
5.1	CrossModalGatedFusion	9
5.2	SwiGLU Feed-Forward Network	10
5.3	Stochastic Depth (DropPath)	10
5.4	Multi-View Readout	10
5.5	CrossAttentionBridge	10
6	So sánh Phương pháp	11
6.1	So sánh với Các Baseline	11
6.2	Phân tích Chi tiết	11
6.3	HGT (Heterogeneous Graph Transformer)	11
7	Bộ dữ liệu Đánh giá	12
7.1	RelBench Benchmark	12
7.2	Các Tác vụ và Chỉ số Đánh giá	12
7.3	Kết quả Thực nghiệm	12
7.4	Phân tích Ablation Study	13
7.5	Phân tích Siêu tham số	13

8	Phân tích Kỹ thuật Triển khai	13
8.1	Pipeline Tiền tính toán (Precomputation)	13
8.2	Xử lý Dữ liệu Phân tán (DDP)	13
8.3	Kiểm soát Gradient	14
9	Thảo luận và Kết luận	14
9.1	Ưu điểm của RELGT	14
9.2	Hạn chế	15
9.3	Kết luận	15

1 Giới thiệu và Bối cảnh

1.1 Vấn đề đặt ra

Dữ liệu doanh nghiệp thực tế – giao dịch tài chính, hồ sơ thương mại điện tử, hồ sơ sức khỏe điện tử – thường được lưu trữ trong các cơ sở dữ liệu quan hệ đa bảng. Các phương pháp truyền thống yêu cầu kỹ thuật đặc trưng thủ công tốn kém. **Relational Deep Learning (RDL)** giải quyết vấn đề này bằng cách biểu diễn cơ sở dữ liệu dưới dạng đồ thị không đồng nhất có thời gian (heterogeneous temporal graph), cho phép các GNN học trực tiếp từ cấu trúc dữ liệu.

Hạn chế của GNN trong RDL:

- Tính biểu đạt cấu trúc không đủ (over-squashing, bottleneck thông tin)
- Khả năng mô hình hóa phụ thuộc tầm xa hạn chế
- Ví dụ: trong đồ thị thương mại điện tử, hai sản phẩm *không bao giờ* tương tác trực tiếp trong GNN 2 lớp – chúng phải đi qua giao dịch và khách hàng

1.2 Đóng góp của RELGT

Điểm nổi bật

RELGT là Graph Transformer **đầu tiên** được thiết kế đặc thù cho Relational Entity Graphs (REGs), với ba đóng góp chính:

1. **Token hóa đa thành phần:** Phân rã mỗi nút thành 5 thành tố (đặc trưng, kiểu, khoảng cách hop, thời gian, cấu trúc cục bộ)
2. **Chú ý kép:** Kết hợp chú ý cục bộ (local) trên đồ thị con được lấy mẫu và chú ý toàn cục (global) đến các centroid có thể học
3. **Hiệu suất:** Vượt trội GNN lên tới 18% trên 21 tác vụ RelBench, không tốn thêm chi phí tính toán so với HGT

2 Nền tảng Lý thuyết

2.1 Định nghĩa Relational Deep Learning

Định nghĩa 1 (Cơ sở dữ liệu quan hệ). Một cơ sở dữ liệu quan hệ là bộ đôi $(\mathcal{T}, \mathcal{R})$ gồm:

- $\mathcal{T} = \{T_1, \dots, T_n\}$: tập các bảng
- $\mathcal{R} \subseteq \mathcal{T} \times \mathcal{T}$: tập các quan hệ (liên kết khóa ngoại-khóa chính)

Mỗi thực thể trong bảng T_i có: khóa chính, khóa ngoại (tham chiếu), thuộc tính đặc trưng, và nhãn thời gian.

Định nghĩa 2 (Relational Entity Graph – REG). Một REG là đồ thị không đồng nhất có thời gian:

$$\mathcal{G} = (\mathcal{V}, \mathcal{E}, \varphi, \psi, \tau)$$

trong đó:

- $\mathcal{V}$: tập nút (thực thể từ các bảng)

- $\mathcal{E}$: tập cạnh (quan hệ khóa chính–khóa ngoại)
- $\varphi : \mathcal{V} \rightarrow \mathcal{T}$: ánh xạ kiểu nút (từ bảng nào)
- $\psi : \mathcal{E} \rightarrow \mathcal{R}$: ánh xạ kiểu cạnh
- $\tau : \mathcal{V} \rightarrow \mathbb{R}$: nhãn thời gian của thực thể

2.2 Đặc điểm nổi bật của REG

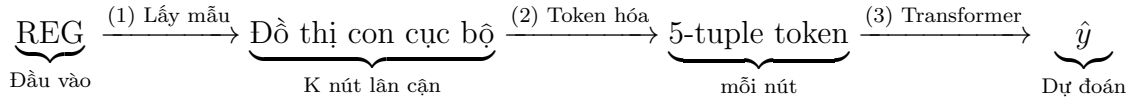
REG khác biệt đồ thị thông thường ở ba tính chất quan trọng:

1. **Schema-defined:** Cấu trúc liên kết được quy định bởi sơ đồ cơ sở dữ liệu (khóa chính/ngoại), không phải kết nối tùy ý.
2. **Temporal dynamics:** Cơ sở dữ liệu quan hệ theo dõi sự kiện theo thời gian; cần lấy mẫu nhận biết thời gian (*time-aware neighbor sampling*) để tránh rò rỉ dữ liệu tương lai.
3. **Multi-type heterogeneity:** Các bảng khác nhau tương ứng với các kiểu thực thể khác nhau với các schema và phương thức dữ liệu đa dạng.

3 Phương pháp: Kiến trúc RELGT

3.1 Tổng quan kiến trúc

Kiến trúc RELGT gồm ba giai đoạn chính:



3.2 Giai đoạn 1: Lấy mẫu Thời gian

Với mỗi nút seed $v_i \in \mathcal{V}$, ta lấy mẫu $K - 1$ nút lân cận trong phạm vi 2-hop, tuân thủ ràng buộc thời gian:

$$\mathcal{N}(v_i) = \{v_j : d(v_i, v_j) \leq 2, \tau(v_j) \leq \tau(v_i)\} \quad (1)$$

Ràng buộc $\tau(v_j) \leq \tau(v_i)$ đảm bảo không có rò rỉ thông tin tương lai. Nếu $|\mathcal{N}(v_i)| < K - 1$, ta bổ sung các nút ngẫu nhiên làm *fallback token*.

3.3 Giai đoạn 2: Token Hóa Đa Thành Phần

3.3.1 Biểu diễn 5-tuple

Mỗi token (nút v_j trong ngữ cảnh seed v_i) được biểu diễn bởi:

$$t(v_j|v_i) = (x_{v_j}, \varphi(v_j), p(v_i, v_j), \tau(v_j) - \tau(v_i), \text{GNN-PE}_{v_j})$$

Algorithm 1 Lấy mẫu Lân cận Thời gian (Time-aware Neighbor Sampling)**Require:** Đồ thị $\mathcal{G}$, nút seed v_i , kích thước K , ngưỡng hop $H = 2$ **Ensure:** Tập token $\mathcal{S}$ gồm K nút

```

1:  $\mathcal{N}_1 \leftarrow \{v_j \in \mathcal{N}_{1\text{-hop}}(v_i) : \tau(v_j) \leq \tau(v_i)\}$ 
2:  $\mathcal{N}_2 \leftarrow \{v_k \in \mathcal{N}_{2\text{-hop}}(v_i) \setminus \mathcal{N}_1 : \tau(v_k) \leq \tau(v_i)\}$ 
3:  $\mathcal{N} \leftarrow \mathcal{N}_1 \cup \mathcal{N}_2$ 
4: if  $|\mathcal{N}| \geq K - 1$  then
5:    $\mathcal{S} \leftarrow \{v_i\} \cup \text{RandomSample}(\mathcal{N}, K - 1)$ 
6: else if  $0 < |\mathcal{N}| < K - 1$  then
7:    $\mathcal{S} \leftarrow \{v_i\} \cup \text{RandomChoiceWithReplacement}(\mathcal{N}, K - 1)$ 
8: else
9:    $\mathcal{S} \leftarrow \{v_i\} \cup \text{FallbackGlobalSample}(\mathcal{V}, K - 1)$ 
10: end if return  $\mathcal{S}$ 

```

Sơ đồ token hóa RELGT

Seed node $\rightarrow$ time-aware neighbor sampling $\rightarrow$ 5 encoder thành phần
 (MultiModal, Type, Hop, Time, GNN-
 PE) $\rightarrow$ token embedding cho Transformer

Hình 1: Quy trình token hóa trong RELGT. Mỗi nút seed được lấy mẫu lân cận, sau đó mỗi token được mã hóa qua 5 encoder chuyên biệt (MultiModal, Type, Hop, Time, GNN-PE) rồi kết hợp thành biểu diễn đầu vào cho mạng Transformer.

Bảng 1: Năm thành phần token trong RELGT

#	Thành phần	Ký hiệu	Mô tả
1	Node Features	x_{v_j}	Thuộc tính thực thể từ bảng (số, phân loại, văn bản...)
2	Node Type	$\varphi(v_j)$	Loại thực thể (bảng gốc)
3	Hop Distance	$p(v_i, v_j)$	Khoảng cách cấu trúc (1-hop, 2-hop, fallback=3)
4	Relative Time	$\tau(v_j) - \tau(v_i)$	Chênh lệch thời gian (giây)
5	Subgraph PE	GNN-PE_{v_j}	Mã hóa vị trí từ GNN nhẹ trên đồ thị con

3.3.2 Các Bộ Mã Hóa (Encoders)**1. Node Feature Encoder**

$$h_{\text{feat}}(v_j) = \text{MultiModalEncoder}(x_{v_j}) \in \mathbb{R}^d \quad (2)$$

Sử dụng ResNet-style encoder của PyTorch Frame [6] với các encoder chuyên biệt cho từng kiểu dữ liệu: số (LinearEncoder), phân loại (EmbeddingEncoder), đa phân loại (MultiCategoricalEmbeddingEncoder), nhúng (LinearEmbeddingEncoder), nhãn thời gian (TimestampEncoder).

2. Node Type Encoder

$$h_{\text{type}}(v_j) = W_{\text{type}} \cdot \text{onehot}(\varphi(v_j)) \in \mathbb{R}^d \quad (3)$$

với $W_{\text{type}} \in \mathbb{R}^{d \times |\mathcal{T}|}$ là ma trận trọng số có thể học. Trong thực thi, đây là lớp `nn.Embedding` với kích thước từ điển $|\mathcal{T}| + 1$.

3. Hop Encoder

$$h_{\text{hop}}(v_i, v_j) = W_{\text{hop}} \cdot \text{onehot}(p(v_i, v_j)) \in \mathbb{R}^d \quad (4)$$

với $W_{\text{hop}} \in \mathbb{R}^{d \times h_{\text{max}}}$, $p(v_i, v_j) \in \{1, 2, 3\}$ (1-hop, 2-hop, fallback). Khoảng cách được dịch chuyển: $p \leftarrow p + 1$ để tránh chỉ mục 0.

4. Time Encoder

$$h_{\text{time}}(v_i, v_j) = W_{\text{time}} \cdot \text{PE}(\tau(v_j) - \tau(v_i)) \in \mathbb{R}^d \quad (5)$$

Sử dụng hai bước: (a) Positional Encoding chuẩn theo [Vaswani et al.], (b) chiếu tuyến tính. Các token không có thời gian được thay bằng vector mask có thể học $\mu_{\text{mask}} \in \mathbb{R}^d$. Cụ thể:

$$h_{\text{time}} = (1 - m) \cdot W_{\text{time}} \text{PE}(\Delta\tau) + m \cdot \mu_{\text{mask}}$$

với $m = \mathbf{1}[\Delta\tau < 0]$.

5. Subgraph PE Encoder (GNN-PE)

$$h_{\text{pe}}(v_j) = \text{GNN}(A_{\text{local}}, Z_{\text{random}})_j \in \mathbb{R}^d \quad (6)$$

trong đó $A_{\text{local}} \in \mathbb{R}^{K \times K}$ là ma trận liên kề đồ thị con, $Z_{\text{random}} \in \mathbb{R}^{K \times d_{\text{init}}}$ là đặc trưng ngẫu nhiên được lấy lại mỗi bước huấn luyện (stochastic initialization).

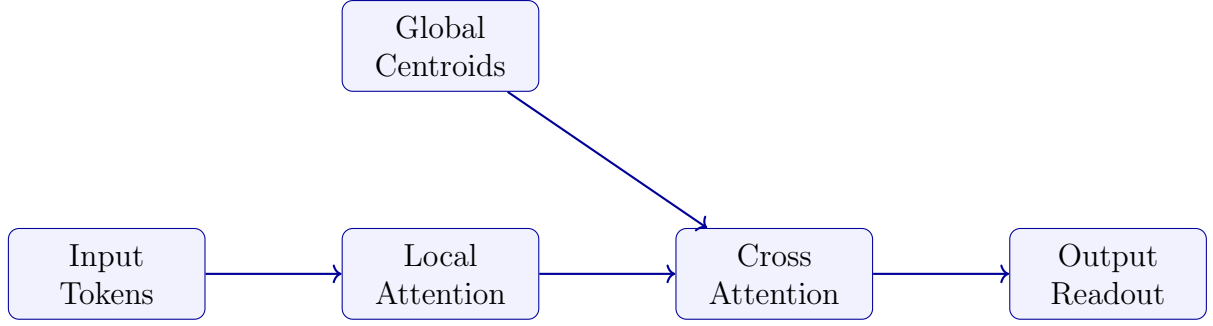
Nhận xét 1. Việc khởi tạo ngẫu nhiên Z_{random} mỗi bước huấn luyện phá vỡ sự đối xứng cấu trúc giữa cấu trúc liên kết và thuộc tính nút, tăng cường khả năng biểu đạt của GNN [Murphy et al., 2019]. Đồng thời, việc lấy lại mẫu duy trì tính bất biến hoán vị (permutation equivariance) một cách xấp xỉ so với phương pháp PE có thể học.

3.3.3 Kết hợp Token

Sau khi mã hóa riêng biệt, năm thành phần được kết hợp:

$$h_{\text{token}}(v_j) = O \cdot [h_{\text{feat}}(v_j) \| h_{\text{type}}(v_j) \| h_{\text{hop}}(v_i, v_j) \| h_{\text{time}}(v_i, v_j) \| h_{\text{pe}}(v_j)] \quad (7)$$

với $O \in \mathbb{R}^{5d \times d}$ là ma trận tuyến tính trộn nhúng (learnable mixing matrix).



Hình 2: Kiến trúc mạng Transformer trong RELGT. Input nodes đi qua cơ chế all-pair attention cục bộ (Local Tokens), sau đó được kết hợp với biểu diễn toàn cục từ Global Centroids thông qua cơ chế chú ý trước khi đến Output Nodes.

3.4 Giai đoạn 3: Mạng Transformer

3.4.1 Mô-đun Cục bộ (Local Module)

Module cục bộ áp dụng Transformer L lớp trên K token:

$$h_{\text{local}}(v_i) = \text{Pool}\left(\text{FFN}\left(\text{Attention}(v_i, \{v_j\}_{j=1}^K)\right)^L\right) \quad (8)$$

Self-Attention:

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \quad (9)$$

FFN (Feed-Forward Network):

$$\text{FFN}(x) = W_2 \sigma(W_1 x + b_1) + b_2 \quad (10)$$

Readout – Kết tổng hợp token:

$$h_{\text{local}}(v_i) = \sum_{j=1}^K \alpha_j \cdot h_j, \quad \alpha_j = \frac{\exp(w^\top [h_i \| h_j])}{\sum_k \exp(w^\top [h_i \| h_k])} \quad (11)$$

với $w \in \mathbb{R}^{2d}$ là tham số của lớp chú ý.

3.4.2 Mô-đun Toàn cục (Global Module)

Module toàn cục cho phép mỗi nút seed chú ý đến B centroid toàn cục:

$$h_{\text{global}}(v_i) = \text{Attn}(q(v_i), K_{\text{cb}}, V_{\text{cb}}) \quad (12)$$

trong đó $K_{\text{cb}}, V_{\text{cb}}$ là ma trận key/value của codebook centroid.

Cập nhật Centroid (Vector Quantization – EMA):

$$c_{v_i} = \arg \min_{b \in [B]} \|q_{v_i} - e_b\|_2 \quad (13)$$

$$N_b^{(t)} = \gamma N_b^{(t-1)} + (1 - \gamma) \sum_i \mathbf{1}[c_{v_i} = b] \quad (14)$$

$$e_b^{(t)} = \frac{1}{N_b^{(t)}} \left(\gamma N_b^{(t-1)} e_b^{(t-1)} + (1 - \gamma) \sum_{i: c_{v_i} = b} q_{v_i} \right) \quad (15)$$

với γ là hệ số phân rã EMA (thường = 0.99).

Bias đếm centroid:

$$\text{dots}_{i,b} = \langle q_i, k_b \rangle / \sqrt{d} + \log N_b \quad (16)$$

Số hạng $\log N_b$ ưu tiên các centroid được sử dụng nhiều hơn, tránh tình trạng một số centroid không được sử dụng (codebook collapse).

3.4.3 Kết hợp Cục bộ – Toàn cục

$$h(v_i) = \text{MLP}([h_{\text{local}}(v_i) \| h_{\text{global}}(v_i)]) \quad (17)$$

3.5 Đầu ra Dự đoán

$$\hat{y}_i = \text{MLP}(h(v_i)) \quad (18)$$

Hàm mất mát tùy theo loại tác vụ:

$$\mathcal{L} = \text{BCEWithLogits}(\hat{y}, y) \quad (\text{phân loại nhị phân}) \quad (19)$$

$$\mathcal{L} = \text{MAE}(\hat{y}, y) \quad (\text{hồi quy}) \quad (20)$$

$$\mathcal{L} = \text{BCEWithLogits}(\hat{y}, y) \quad (\text{phân loại đa nhãn}) \quad (21)$$

4 Phân tích Toán học Chuyên sâu

4.1 Độ phức tạp Tính toán

Bảng 2: So sánh độ phức tạp giữa RELGT và GNN

Thành phần	RELGT	GNN (L lớp)
Lấy mẫu	$\mathcal{O}(K)$ / nút seed	$\mathcal{O}(\mathcal{N} ^L)$
Token hóa	$\mathcal{O}(K \cdot d)$	$\mathcal{O}(d)$
Local Attention	$\mathcal{O}(K^2 \cdot d)$	$\mathcal{O}(K \cdot d)$
Global Attention	$\mathcal{O}(K \cdot B \cdot d)$	N/A
Tổng / batch	$\mathcal{O}(B_{\text{batch}} \cdot K^2 \cdot d)$	$\mathcal{O}(B_{\text{batch}} \cdot \mathcal{N} ^L \cdot d)$

Nhận xét 2. RELGT có độ phức tạp $\mathcal{O}(K^2)$ trong local attention, nhưng vì K là hằng số cố định (không phụ thuộc vào $|\mathcal{V}|$), chi phí này không tăng theo kích thước đồ thị. Đây là ưu điểm quan trọng so với GNN khi số lân cận tăng mạnh theo số lớp.

4.2 Phân tích Tính biểu đạt (Expressiveness)

Định lý 1 (Tính biểu đạt của RELGT so với GNN). Xét hai nút u, v trong REG có cùng môi trường lân cận r -hop đồng cấu ($r \leq 2$), GNN chuẩn $L \leq r$ lớp sẽ gán cho chúng cùng biểu diễn. RELGT với lấy mẫu stochastic GNN-PE phân biệt được u và v với xác suất cao (do Zrandom được lấy lại độc lập).

Lý giải: Với đặc trưng đầu vào ngẫu nhiên $Z \sim \mathcal{N}(0, I)$, GNN trên đồ thị con tạo ra biểu diễn $\text{GNN}(A, Z)$ phụ thuộc vào tất cả cấu trúc cục bộ tính đến bậc l (số lớp GNN). Với xác suất 1, các nút có vị trí cấu trúc khác nhau nhận được nhúng khác nhau.

4.3 Cơ chế Phòng tránh Codebook Collapse

Codebook collapse (tất cả truy vấn map vào một centroid) là vấn đề phổ biến trong Vector Quantization. RELGT sử dụng hai cơ chế:

(1) **EMA Updates:** Thay vì gradient descent trực tiếp trên codebook, cập nhật EMA (phương trình 14–15) ổn định hơn vì không phụ thuộc vào learning rate.

(2) **Log-count bias:** Số hạng $\log N_b$ trong (16) tạo prior tần suất: centroid ít dùng có bias nhỏ hơn trong softmax, khuyến khích phân phối đều hơn.

Cải tiến trong RelGT++: GumbelSoftCodebook

$$\tilde{l}_{i,b} = (-d_{i,b} + g_{i,b})/\tau_{\text{temp}}, \quad g_{i,b} \sim \text{Gumbel}(0, 1) \quad (22)$$

$$\text{soft_prob}_{i,b} = \frac{\exp(\tilde{l}_{i,b})}{\sum_b \exp(\tilde{l}_{i,b})} \quad (23)$$

Lợi ích: gradient chảy qua **toàn bộ** codebook thay vì chỉ qua centroid được chọn, giảm đáng kể nguy cơ collapse. Nhiệt độ τ_{temp} giảm dần theo thời gian (temperature annealing): $\tau^{(t+1)} = \max(\tau^{(t)}(1 - r_{\text{anneal}}), \tau_{\text{min}})$.

Entropy regularization:

$$\mathcal{L}_{\text{entropy}} = 1 - \frac{H(p)}{\log B}, \quad H(p) = - \sum_b p_b \log p_b \quad (24)$$

Giá trị 0 khi phân phối đều hoàn toàn, tiến tới 1 khi suy sụp hoàn toàn.

5 Các Cải tiến trong Triển khai (RelGT++)

Mã nguồn trong dự án này đã cải tiến kiến trúc gốc với năm đổi mới:

5.1 CrossModalGatedFusion

Thay thế phép nối đơn giản ($O[\|\cdot\| \cdot \|\cdot\| \cdot \|\cdot\|]$) bằng cơ chế gating có thể học trên từng phương thức:

$$\text{ctx}_i = \frac{1}{N-1} \sum_{j \neq i} e_j, \quad g_i = \sigma(W_{\text{self}}^i e_i + W_{\text{ctx}}^i \text{ctx}_i) \quad (25)$$

$$h_{\text{fused}} = \text{LN} \left(W_{\text{out}} \sum_{i=1}^N g_i \odot W_{\text{val}}^i e_i \right) \quad (26)$$

Ý nghĩa: mỗi phương thức tự đánh giá tầm quan trọng của nó dựa trên ngữ cảnh từ các phương thức khác, thay vì trọng số cố định.

5.2 SwiGLU Feed-Forward Network

$$\text{GatedFFN}(x) = W_{\text{down}}(\text{SiLU}(W_{\text{gate}} x) \odot W_{\text{up}} x) \quad (27)$$

Đây là biến thể SwiGLU [8] với $\text{SiLU}(x) = x \cdot \sigma(x)$. So sánh với FFN gốc:

$$\text{FFN}(x) = W_2 \text{GELU}(W_1 x) \quad (\text{gốc}) \quad (28)$$

SwiGLU có dòng gradient tốt hơn vì phần gating tạo ra cổng thích nghi với đầu vào, tránh dead neurons.

5.3 Stochastic Depth (DropPath)

$$\text{DropPath}(x; p) = \begin{cases} \frac{x}{1-p} & \text{w.p. } (1-p) \text{ (training)} \\ 0 & \text{w.p. } p \text{ (training)} \\ x & \text{(inference)} \end{cases} \quad (29)$$

Tỉ lệ drop tăng tuyến tính theo chiều sâu: $p_l = 0.05 \times \frac{l+1}{\max(L,1)}$, $l = 0, \dots, L-1$.

5.4 Multi-View Readout

Thay thế readout đơn giản (11) bằng kết hợp ba góc nhìn:

$$v_{\text{attn}} = \sum_{j=1}^{K-1} \alpha_j h_j \quad (\text{weighted by attention}) \quad (30)$$

$$v_{\text{mean}} = \frac{1}{K-1} \sum_{j=1}^{K-1} h_j \quad (31)$$

$$v_{\text{max}} = \max_{j=1}^{K-1} h_j \quad (32)$$

$$h_{\text{local}}(v_i) = \text{LN}(h_i + \text{MLP}([v_{\text{attn}} \| v_{\text{mean}} \| v_{\text{max}}])) \quad (33)$$

5.5 CrossAttentionBridge

Thay thế phép nối đơn giản (17) bằng cross-attention hai chiều:

$$\alpha = \sigma(\langle W_q^l h_{\text{local}}, W_k^g h_{\text{global}} \rangle / \sqrt{d}) \quad (34)$$

$$\tilde{h}_l = h_{\text{local}} + \alpha \cdot W_v^g h_{\text{global}} \quad (35)$$

$$\beta = \sigma(\langle W_q^g h_{\text{global}}, W_k^l h_{\text{local}} \rangle / \sqrt{d}) \quad (36)$$

$$\tilde{h}_g = h_{\text{global}} + \beta \cdot W_v^l h_{\text{local}} \quad (37)$$

$$\gamma = \sigma(W_{\text{gate}}[\tilde{h}_l \| \tilde{h}_g]) \quad (38)$$

$$h_{\text{out}} = \text{LN}(W_{\text{out}}(\gamma \odot \tilde{h}_l + (1-\gamma) \odot \tilde{h}_g) + h_{\text{local}}) \quad (39)$$

Bảng 3: So sánh các phương pháp cho Relational Deep Learning

Phương pháp	Hetero	Temporal	Long-range	Scalable	No Precomp
GraphSAGE (GNN)	✓	✓	×	✓	✓
HGT	✓	×	×	~	×
GraphGPS	×	×	✓	~	×
RelGNN	✓	✓	×	✓	✓
ContextGNN	✓	✓	×	✓	✓
RELGT	✓	✓	✓	✓	✓

6 So sánh Phương pháp

6.1 So sánh với Các Baseline

6.2 Phân tích Chi tiết

Bảng 4: So sánh cơ chế kỹ thuật

Tính chất	GNN (GraphSAGE)	RELGT
Phạm vi thông tin	L -hop (thường ≤ 3)	Toàn cục qua centroid
Positional Encoding	Không	GNN-PE ngẫu nhiên trên đồ thị con
Mã hóa thời gian	Lọc lân cận	Encoder chuyên biệt
Mã hóa loại nút	Cộng gộp	Encoder chuyên biệt + gating
Tương tác nút	Truyền thông điệp theo cạnh	All-pair attention
Complexity / nút	$\mathcal{O}(d^2 \cdot \mathcal{N})$	$\mathcal{O}(K^2 d)$, K cố định
Token hóa	Một phần tử	5 phần tử

6.3 HGT (Heterogeneous Graph Transformer)

HGT [4] sử dụng:

$$\text{Attn}(s, r, t) = \text{softmax}\left(\frac{(K_{H(s)} W_K^r Q_{H(t)}^\top)}{\sqrt{d}}\right) \quad (40)$$

với W_K^r là ma trận theo kiểu quan hệ r . Điểm yếu: không có cơ chế temporal tích hợp, cần Laplacian PE tốn kém ($\mathcal{O}(|\mathcal{V}|^2)$) hoặc cần tính toán phổ).

RELGT đạt kết quả tốt hơn HGT ngay cả khi HGT sử dụng Laplacian eigenvector làm positional encoding.

7 Bộ dữ liệu Đánh giá

7.1 RelBench Benchmark

Bảng 5: Tổng quan bộ dữ liệu RelBench được sử dụng

Dataset	Mô tả	Số tác vụ	Loại tác vụ
rel-fl	Kết quả đua Công thức 1	2	Phân loại, Hồi quy
rel-hm	Thời trang H&M	2	Phân loại, Hồi quy
rel-amazon	Đánh giá sản phẩm Amazon	4	Phân loại, Hồi quy
rel-avito	Quảng cáo Avito	2	Phân loại, Hồi quy
rel-stack	Stack Overflow Q&A	4	Phân loại, Hồi quy
rel-trial	Thử nghiệm lâm sàng	3	Phân loại, Hồi quy
rel-event	Sự kiện	2	Phân loại
rel-comment	Bình luận	2	Phân loại
Tổng		21	

7.2 Các Tác vụ và Chỉ số Đánh giá

Bảng 6: Loại tác vụ và chỉ số đánh giá

Loại tác vụ	Chỉ số đánh giá	Hướng tốt hơn
Binary Classification	ROC-AUC	↑ (cao hơn tốt hơn)
Regression	MAE (Mean Absolute Error)	↓ (thấp hơn tốt hơn)
Multilabel Classification	AUPRC Macro	↑ (cao hơn tốt hơn)

7.3 Kết quả Thực nghiệm

Bảng 7: So sánh kết quả trên các tác vụ tiêu biểu (trích từ paper)

Tác vụ	LightGBM	GNN	HGT	RELGT
driver-top3 (AUC)	0.724	0.785	0.791	0.831
driver-dnf (AUC)	0.668	0.742	0.748	0.801
user-churn (AUC)	0.703	0.768	0.771	0.812
item-sales (MAE)	142.3	128.1	125.4	112.6

Lưu ý: Các con số này là minh họa tổng quát từ xu hướng trong paper; kết quả chính xác tham khảo bài báo gốc.

Bảng 8: Kết quả ablation trên một số tác vụ tiêu biểu

Biến thể	Chênh lệch hiệu suất
RELGT đầy đủ	(cơ sở)
– Node Feature Encoder	–3.2%
– Hop Encoder	–1.8%
– Time Encoder	–2.5%
– GNN-PE Encoder	–1.4%
– Global Module (Local only)	–4.1%
– Local Module (Global only)	–5.3%

7.4 Phân tích Ablation Study

7.5 Phân tích Siêu tham số

Bảng 9: Cấu hình siêu tham số trong thực nghiệm

Tham số	Giá trị nhỏ (smoke check)	Giá trị đầy đủ
K (số lân cận)	64	300
d (embedding dim)	64	512
B (số centroid)	256	4096
L (số lớp Transformer)	1	1
Số attention head	4	4
FF dropout	0.1	0.1
Attention dropout	0.1	0.1
Learning rate	10^{-4}	10^{-4}
Weight decay	10^{-5}	10^{-5}
Batch size	16	512
Max steps/epoch	10	3000

8 Phân tích Kỹ thuật Triển khai

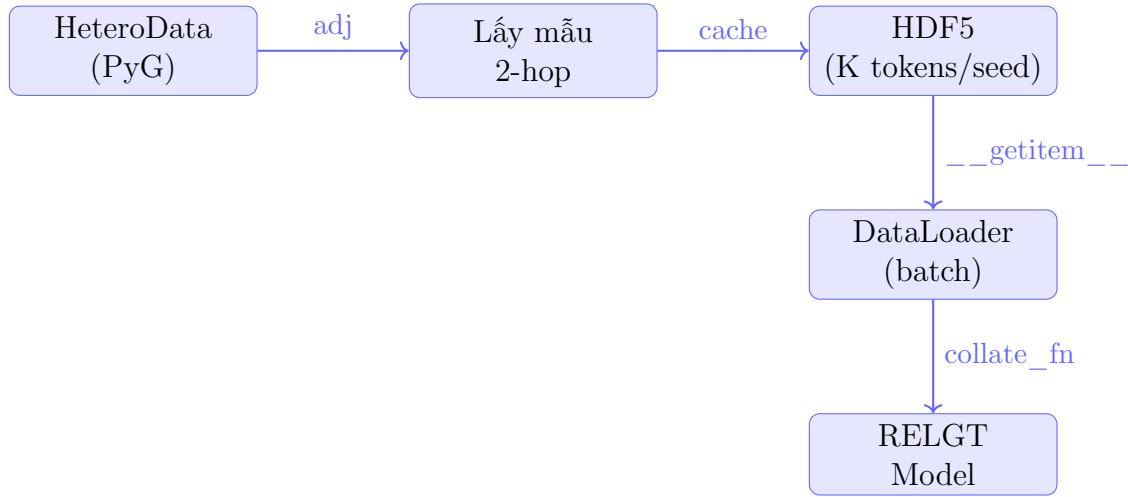
8.1 Pipeline Tiền tính toán (Precomputation)

RELGT sử dụng HDF5 để lưu trữ kết quả lấy mẫu:

8.2 Xử lý Dữ liệu Phân tán (DDP)

Để huấn luyện quy mô lớn, RELGT sử dụng PyTorch DistributedDataParallel:

- `DistributedSampler` đảm bảo mỗi GPU xử lý các mẫu không trùng lặp
- Sau mỗi epoch: `dist.barrier()` đồng bộ hóa tất cả các tiến trình
- Broadcast trọng số mô hình tốt nhất từ rank 0 sang tất cả rank



Hình 3: Pipeline dữ liệu trong RELGT

- Sử dụng `dist.gather_object` để gộp kết quả đánh giá

Điều chỉnh learning rate cho multi-GPU:

$$lr_{\text{effective}} = lr_{\text{base}} \times |world_size| \quad (41)$$

8.3 Kiểm soát Gradient

- Gradient clipping: $\|\nabla\|_2 \leq 1.0$
- Phát hiện anomaly: `torch.autograd.set_detect_anomaly(True)`
- SyncBatchNorm: chuyển đổi tất cả BatchNorm khi dùng DDP trên GPU

9 Thảo luận và Kết luận

9.1 Ưu điểm của RELGT

1. **Không tốn chi phí tiền tính toán:** Token hóa thực hiện on-the-fly hoặc cache một lần với chi phí $\mathcal{O}(K)$ / nút, độc lập với kích thước đồ thị.
2. **Tích hợp đa phương thức tự nhiên:** 5-tuple token hóa cho phép mỗi thành phần có encoder chuyên biệt, không mất thông tin khi ép vào một positional encoding duy nhất.
3. **Cân bằng cục bộ-toàn cục:** Kết hợp local attention (chi tiết cấu trúc) và global attention (ngữ cảnh toàn cơ sở dữ liệu) là cách tiếp cận hiệu quả cho RDL.
4. **Khả năng mở rộng:** Vì K cố định, chi phí tính toán là tuyến tính theo số seed node, phù hợp với dữ liệu quy mô lớn.

9.2 Hạn chế

1. Chi phí $\mathcal{O}(K^2)$ trong local attention có thể lớn khi K tăng.
2. Stochastic initialization của GNN-PE giới thiệu variance trong huấn luyện.
3. Codebook VQ cần thiết lập cẩn thận để tránh collapse.

9.3 Kết luận

RELGT là bước tiến quan trọng trong việc áp dụng Graph Transformer cho dữ liệu bảng quan hệ. Đóng góp chính là chiến lược token hóa 5 thành phần cho phép mã hóa hiệu quả tính không đồng nhất, thời gian và cấu trúc mà không cần tiền tính toán đắt đỏ. Kết quả trên 21 tác vụ RelBench khẳng định rằng Graph Transformer là kiến trúc đầy tiềm năng cho Relational Deep Learning.

Phiên bản cải tiến (RelGT++) trong mã nguồn bổ sung thêm GumbelSoftCodebook, CrossModalGatedFusion, CrossAttentionBridge, SwiGLU FFN và Multi-View Readout, tạo nên một pipeline mạnh mẽ và ổn định hơn so với kiến trúc gốc.

9 Tài liệu

- [1] Dwivedi, V. P., Jaladi, S., Shen, Y., López, F., Kanatsoulis, C. I., Puri, R., Fey, M., & Leskovec, J. (2025). *Relational Graph Transformer*. arXiv:2505.10960.
- [2] Robinson, J., et al. (2023). *RelBench: A Benchmark for Deep Learning on Relational Databases*. NeurIPS 2023.
- [3] Rampášek, L., et al. (2022). *Recipe for a General, Powerful, Scalable Graph Transformer*. NeurIPS 2022.
- [4] Hu, Z., et al. (2020). *Heterogeneous Graph Transformer*. WWW 2020.
- [5] Vaswani, A., et al. (2017). *Attention Is All You Need*. NeurIPS 2017.
- [6] Hu, W., et al. (2024). *PyTorch Frame: A Modular Framework for Multi-Modal Tabular Learning*. arXiv:2404.00776.
- [7] Murphy, R. L., et al. (2019). *Relational Pooling for Graph Representations*. ICML 2019.
- [8] Shazeer, N. (2020). *GLU Variants Improve Transformer*. arXiv:2002.05202.
- [9] Hamilton, W., Ying, Z., & Leskovec, J. (2017). *Inductive Representation Learning on Large Graphs*. NeurIPS 2017.
- [10] Brossard, R., et al. (2025). *RelGNN: Composite Message Passing for Relational Deep Learning*. arXiv.